DOKUZ EYLUL UNIVERSITY

ENGINEERING FACULTY

DEPARTMENT OF COMPUTER ENGINEERING

CME2210 OBJECT ORIENTED ANALYSIS AND DESIGN

REPORT

Beauty System

by

Şengül Akkaya 2023510121

Arzu Hamdioğlu 2023510054

Zeliha Erdoğan 2023510145

Lecturer

Prof. Dr. Semih Utku

IZMIR

18.03.2026

OUTLINE

1. System Definition and Objectives
2. Functional Requirements
3. Non-Functional Requirements
4. Use Cases
5. Use Case Diagram

1. System Definition and Objectives

System Definition

“CenGIRL – Beauty Salon Management System” is a comprehensive software solution designed to digitize and optimize traditional, manual appointment, staff, and service management processes. The system addresses operational challenges such as appointment conflicts, the complexity of staff work schedules, and the lack of structure in payment processes. It enables customers to view available time slots and create online appointments, allows employees to manage their own calendars and service statuses, and empowers salon managers to control service catalogs, staff records, and payroll from a single central hub.

Project Objectives

In line with the academic requirements of this semester's project, the primary objectives of developing this system are:

- To prioritize and maintain the highest level of software design quality, rather than focusing on a graphical user interface (GUI) or persistent database implementation.
- To design and implement a medium-scale system using Java , utilizing entirely in-memory data structures for data persistence.
- To flawlessly implement Object-Oriented Analysis and Design (OOAD) principles and Clean Code practices.
- To establish a strict Layered (N-Tier) architecture consisting of Presentation , Application/Service , Domain , and Infrastructure layers.
- To build a robust, flexible, and sustainable backend framework that is fully compliant with SOLID principles and successfully integrates multiple design patterns.

2. Functional Requirements

The core functions the system must perform through the designated actors (Customer, Employee, Manager) and background services are defined modularly below:

A. Customer (Actor) Functions

Authentication and Profile Management:

- Users should be able to create a new registration in the system (Register).
- Registered users should be able to log in to the system (Log in). The system must verify identity (Verify Identity) at this stage and securely manage incorrect login attempts (Handle wrong log in).
- Users should be able to view and update their own profile information (View/Update profile).

Appointment and Service Management:

- Users should be able to list all services offered by the salon (List Services).
- Users should be able to query available time slots for a specific service and employee (View Available Time Slots).
- Users should be able to create a new appointment by selecting an available time slot (Create Appointment). This process must automatically trigger the "Notification Service".
- Users should be able to view the details of their past appointments (View Past Appointments).
- Active appointments should be cancellable (Cancel Appointment). The cancellation process must update the relevant employee's calendar and send a notification via the "Notification Service".

Payment Operations:

- Users should be able to make payments for the services they received through the system (Make Payment). This process must utilize the "Payment Service" infrastructure.

B. Manager Functions

Service and Employee Management:

- The manager should be able to add, update, and delete the beauty services offered in the system (name, duration, price) (Service Management).

- New employees should be able to be added to the system, and the records of existing employees should be managed (Employee Management).
- The manager should be able to set employees' working hours, days off, and availability (Manage Working Hours).

Financial Transactions:

- The manager should be able to calculate and process employees' salaries (Pay Salary). This function must be integrated with the "Payment Service."

C. Employee Functions

Calendar and Task Tracking:

- Employees should be able to log in to the system using their unique credentials (Employee Log in).
- Employees should be able to view their assigned appointments and daily/weekly work schedules (View My Calendar).
- The status of completed, canceled, or no-show appointments should be updatable by the employee (Update Service Status).

D. Core System Services

Notification Service:

- Responsible for sending status updates to the relevant actors (customer and employee) during the appointment creation and cancellation processes.

Payment Service:

- The centralized business logic responsible for both receiving service payments from customers and processing salary payments executed by the manager.

3. Non-Functional Requirements

The fundamental constraints determining the system's quality, architectural robustness, and compliance with software engineering standards are listed below:

- **Maintainability:** The primary focus of the system is software design quality, rather than UI or database implementation. The codebase will be built strictly adhering to Clean Code practices and SOLID principles. By utilizing a layered architecture, responsibilities will be strictly separated across the Presentation, Application/Service, Domain, and Infrastructure layers. This separation will highly facilitate code readability, debugging processes, and overall maintenance.

- **Extensibility:** In accordance with the Open-Closed Principle, the core business logic will remain unmodified when new service types, payment methods, or employee roles are introduced to the system. This flexibility will be achieved by strategically integrating various Design Patterns (such as Strategy or Factory Method).

- **Data Integrity:** Although the system will use in-memory repositories instead of a persistent database, referential consistency among objects must be strictly maintained. For instance, when an appointment is canceled, the system must immediately make that time slot "available" again and synchronously update the employee's calendar. To prevent data inconsistencies like double-booking, object states within the Domain Layer will be strictly protected through encapsulation rules.

- **Security:** Authorization boundaries will be clearly defined during the *Log in* and *Verify Identity* processes for all actors (Manager, Employee, Customer). Within the boundaries of object-oriented design, an actor's unauthorized access

to restricted data (e.g., a customer accessing other customers' payment details or the salon's revenue) will be prevented using strict access modifiers and role-based service validations.

- **Performance:** Considering that all operations and repositories will be executed in-memory, optimum time complexity will be targeted for data searching, filtering (e.g., finding available time slots), and listing operations. To achieve this, the most appropriate data structures (e.g., HashMaps, HashSets) will be carefully selected when managing collections.

- **Availability & Robustness:** The system must not crash upon invalid or incorrect data entries (e.g., entering a date in the wrong format or incorrect password attempts). A comprehensive Exception Handling mechanism will be implemented to control runtime errors and ensure the system remains operational and stable under all circumstances.

- **Usability**: Since the system will be tested via the console, the menu flows, process guidance, and error messages presented to the user will be designed to be highly intuitive and easy to understand.

4.Use Cases

Use Case 1: Create Appointment

Primary Actor: Customer
Included Use Cases: Notification Service
Main Success Scenario:

1. The customer selects a specific beauty service and an employee from the system catalog.
2. The system retrieves and displays the available time slots for the selected employee.
3. The customer chooses a suitable time slot and confirms the appointment request.
4. The system saves the appointment with a "Scheduled" status and updates the employee's calendar.
5. The system triggers the **Notification Service** to send a booking confirmation to both parties.

Use Case 2: Complete Service and Make Payment

Primary Actor: Employee & Customer

Included Use Cases: Payment Service

Main Success Scenario:

1. The employee selects the scheduled appointment from their calendar and updates the service status to "Completed".
2. The system calculates the final fee for the provided service and generates a pending payment record.
3. The customer selects the "Make Payment" option from their profile for the completed appointment.
4. The system triggers the **Payment Service** to process the transaction securely.
5. Upon success, the system updates the appointment status to "Paid" and finalizes the transaction.

Use Case 3: Cancel Appointment

Primary Actor: Customer

Included Use Cases: Notification Service

Main Success Scenario:

1. The customer selects an active appointment from their history and requests a cancellation.
2. The system prompts for a final confirmation, and the customer approves.
3. The system updates the appointment status to "Cancelled" in the in-memory repository.
4. The system immediately frees the previously reserved time slot in the employee's calendar.
5. The system triggers the **Notification Service** to inform the assigned employee about the cancellation.

Use Case 4: Pay Salary

Primary Actor: Manager

Included Use Cases: Payment Service

Main Success Scenario:

1. The manager navigates to the payroll module and selects an employee from the list.
2. The manager enters or confirms the standard monthly salary amount for the selected employee.
3. The system triggers the **Payment Service** to process the transaction.
4. The system updates the employee's financial record to "Paid" for the current month.
5. The system displays a transaction success message to the manager.

Use Case Diagram

Beauty Salon Management System

